

DATA 266 - Homework 5 Findings

Fine-Tuning an LLM Using LoRA (flan-t5-small on DialogSum)

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

SEED 9486 is used for Python, NumPy, PyTorch (including CUDA), and transformers, and is also passed to the Trainer as seed and data_seed and used to shuffle the training split. SLICE picks the first evaluation dialogue (test index 486). HP_ID, CLS_A, and CLS_B do not map to anything in a summarization task, so they are only reported for consistency.

Introduction

For this assignment I fine-tuned google/flan-t5-small, an encoder-decoder model with about 77M parameters, to summarize everyday conversations from DialogSum (neil-code/dialogsum-test) using LoRA through the Hugging Face PEFT library. I started from the professor's Demo_5_LoRA.ipynb and kept its overall flow, but the demo is a causal language model setup (Mistral-7B, 8-bit quantization, DataCollatorForLanguageModeling), so the main work was adapting it correctly to sequence-to-sequence summarization. Every number and output in this report comes from the executed HW5_LoRA_Fine_Tuning.ipynb notebook, run on Google Colab with a Tesla T4 GPU, in a single Run all in a fresh runtime.

What's in this submission

Task	What was completed
Task 1	Loaded and inspected DialogSum, converted each row to input (dialogue) and target (summary) with preprocess_dialogsum, tokenized the dialogue as encoder input and the summary as decoder labels, and printed two processed samples.
Task 2	Loaded flan-t5-small and generated baseline summaries for two held-out test dialogues with greedy decoding.
Task 3	Attached LoRA adapters (r=4, alpha=16, dropout=0.1) to the T5 q and v projections, after checking those module names exist, and printed total and trainable parameter counts.
Task 4	Fine-tuned with Seq2SeqTrainer and DataCollatorForSeq2Seq and saved the adapter to outputs/dialogsum_lora_r4.
Task 5	Reloaded the saved adapter onto a fresh base model and summarized the same two dialogues with the same generation settings.
Task 6	Comparison table of reference, baseline, and LoRA outputs, with observations.
Task 7	Trained a second adapter (r=16, alpha=32) under the same budget, saved it separately to outputs/dialogsum_lora_r16, and compared the two ranks.

Task 1: Data Preparation

DialogSum pairs multi-turn conversations with short human-written summaries. flan-t5 is instruction-tuned, so each dialogue is wrapped in a short prompt ("Summarize the following conversation. ... Summary:"), and the same prompt is used for training and for every inference call. This is a sequence-to-sequence task: the dialogue goes into the encoder, and the decoder is trained to produce only the summary. Labels are padded with -100 by the collator so padding does not count toward the loss.

Setting	Value
Splits	train 1,999, validation 499, test 499 (columns id, dialogue, summary, topic)
Training subset	1,000 examples from the train split, shuffled with SEED, to keep both training runs short on a T4
Evaluation dialogues	test[486] (test_162_1) and test[489] (test_163_1)
Why not 486 and 487	the test split repeats each dialogue in consecutive rows, once per reference summary, so 487 is the same conversation as 486; the notebook takes the next row with a different dialogue and asserts that the two differ
Leakage check	neither evaluation dialogue appears in the training subset (assertion passed)
Max source / target length	512 / 128 tokens, truncation only, dynamic padding

Task 3: Applying LoRA

The demo targets `q_proj` and `v_proj`, which are Mistral's layer names. In T5 the attention projections are just `q`, `k`, `v`, and `o`, so the notebook lists the model's linear layers first and confirms that `q` and `v` each appear in 24 attention layers before building the LoRA config (task_type SEQ_2_SEQ_LM).

LoRA freezes every original weight matrix W and learns a low-rank update instead, so a targeted layer computes $W \times x + (\alpha / r) B A x$, where A is r by d_{in} and B is d_{out} by r . For a 512 by 512 projection at $r=4$ that is 4,096 trainable values instead of 262,144. B starts at zero, so training starts from exactly the base model. Only A and B get gradients, and everything else (embeddings, feed-forward layers, the `k` and `o` projections, the LM head) stays frozen, which is why the trainable share is so small.

Config	Total parameters	Trainable parameters	Trainable %	Adapter size
<code>r=4, alpha=16, dropout=0.1</code>	77,133,184	172,032	0.2230%	684.8 KB
<code>r=16, alpha=32, dropout=0.1</code>	77,649,280	688,128	0.8862%	2,701.1 KB

Task 4: Fine-Tuning

Both adapters were trained with exactly the same budget and settings. `fp16` is off because T5 models are known to overflow in `fp16`, and at this model size `fp32` is still fast on a T4. The learning rate is higher than the demo's $2e-4$ because a small model with low-rank adapters needs a larger step size to move noticeably in 250 steps.

Setting	Value
Training examples / epochs / batch size	1,000 / 2 / 8 (250 optimizer steps)
Learning rate	$1e-3$ with 10 warmup steps, weight decay 0.0
Precision	<code>fp32</code> , no quantization

Setting	Value
Trainer / collator	Seq2SeqTrainer / DataCollatorForSeq2Seq (label_pad_token_id = -100)
Seed	9486 (seed and data_seed)
Device	cuda, Tesla T4 on Google Colab (fresh runtime)

Training loss

Step	r=4 loss	r=16 loss
25	2.1567	2.0876
50	1.6888	1.6578
75	1.5910	1.5660
100	1.5677	1.5505
125	1.6214	1.6044
150	1.5611	1.5348
175	1.5971	1.5654
200	1.5550	1.5265
225	1.4628	1.4365
250	1.5171	1.4858
Final (average over run)	1.6319	1.6015
Runtime	66.8 s	64.6 s

The loss dropped quickly in the first 50 steps and then went down slowly for the rest of the run for both ranks. r=16 was a little lower at every logged step, which makes sense given it has four times the trainable parameters to fit the training data with.

Tasks 2, 5, and 6: Before and After Fine-Tuning

All outputs use greedy decoding (do_sample=False, num_beams=1, max_new_tokens=100), so each output depends only on the model weights. Both adapters were reloaded from disk onto a fresh copy of flan-t5-small and merged before generating.

Example 1 (test index 486, test_162_1)

```
#Person1#: Ernie, I'm really excited about starting this band together. We're going to be the
two coolest students on campus.
#Person2#: I know! It's gonna be great. But what kind of music do you think we should play?
#Person1#: That's a good question. You know I love hip hop, so maybe we can play some Vanilla
Ice songs.
#Person2#: Hmm. . . Vanilla Ice? I know his most famous song, ' Ice Ice Baby, ' but I don't
know his other songs.
#Person1#: That's OK. You can buy his songs on iTunes and listen to them.
```

	Summary
Reference	#Person1# and Ernie plan to start a band and they decide to play hip hop music.
Baseline	What's the name of the band?
LoRA r=4	Ernie and Ernie are excited about starting the band together. They're going to be the two coolest students on campus. They're going to play some Vanilla Ice songs.

Example 2 (test index 489, test_163_1)

```
#Person1#: Are you enjoying your trip to New Orleans?
#Person2#: Oh, yes. I really like it here.
#Person1#: Would you like to do something tonight?
#Person2#: Sure. I'd love to.
#Person1#: Let's see. Have you been to a jazz club yet?
#Person2#: Yes. I've already been to several clubs here.
#Person1#: OK. What about an evening riverboat tour?
#Person2#: Uh, actually, I've gone twice this week.
#Person1#: So, what do you want to do?
#Person2#: Well, I haven't been to the theater in a long time.
#Person1#: Oh, OK. I hear there's a terrific show at the Sanger Theater.
#Person2#: Great! Let's make a reservation.
```

	Summary
Reference	#Person1# and #Person2# are discussing where to have fun, and they decide to go to the theater tonight.
Baseline	How about a riverboat tour?
LoRA r=4	#Person2# wants to go to New Orleans to see jazz club. #Person2# hasn't been to the theater in a long time.

What changed

The clearest change is the format. Before fine-tuning, the base model did not summarize either conversation and just returned a question copied out of the dialogue. After LoRA fine-tuning, both outputs read like DialogSum summaries, in the third person and using the #Person# tags from the training data.

The content is only partly right though. Example 1 gets the main idea (starting a band, playing Vanilla Ice songs) but calls both speakers "Ernie" and copies "the two coolest students on campus" almost straight from the dialogue. Example 2 picks up that #Person2# hasn't been to the theater in a long time, but it gets the setup wrong (they are already in New Orleans, and the jazz club was turned down) and never says that they decide to go to the theater, which is the main point of the reference. So 250 steps of LoRA training mostly taught the model what a summary should look like, more than it improved its understanding of the conversation.

Task 7: LoRA Rank Experiment (r=4 vs r=16)

The only things that changed between the two runs are the rank and lora_alpha, as the assignment specifies. The base model, tokenizer, training subset, preprocessing, seed, epochs, batch size, learning rate, and generation settings are the same. One side effect is that the effective scaling α / r is lower for r=16 ($32 / 16 = 2$) than for r=4 ($16 / 4 = 4$).

Rank	Trainable params	Trainable %	Final loss	Example 1 output	Example 2 output
4	172,032	0.2230%	1.6319	Ernie and Ernie are excited about starting the band together. They're going to be the two coolest students on campus. They're going to play some Vanilla Ice songs.	#Person2# wants to go to New Orleans to see jazz club. #Person2# hasn't been to the theater in a long time.
16	688,128	0.8862%	1.6015	Ernie and Ernie are excited about starting the band together. Ernie and Ernie are excited about the music they should play.	#Person2# and #Person1# are going to New Orleans tonight. #Person2# wants to go to a jazz club and a riverboat tour.

r=16 had four times the trainable parameters and a slightly lower training loss, but that did not lead to better summaries on these two examples. On Example 1, r=16 repeats "Ernie and Ernie are excited" twice and drops Vanilla Ice, which is the closest thing to the hip hop detail in the reference, while r=4 kept it. On Example 2, r=16 mentions both speakers but says #Person2# wants the jazz club and the riverboat tour, the two options that were actually rejected, and it never mentions the theater. r=4 was also wrong about the setup but at least mentioned the theater.

My conclusion is that rank was not the limiting factor for this setup. Both adapters learned the same summary style and made similar kinds of mistakes, and the r=16 model's better fit on the training data did not show up as more accurate summaries. Two examples are not enough to say r=4 is better in general, and I did not compute a numerical metric like ROUGE, but at this training budget r=4 gave similar or slightly better outputs with a quarter of the trainable parameters.

A note on reproducibility

I ran the final notebook on Colab twice (once in a reused runtime and once in a fresh runtime, which is the submitted run), and both gave the same parameter counts and word for word the same summaries. A few losses differed in the fourth decimal place (r=4 final loss 1.6318 vs 1.6319), which is expected because some GPU kernels do not add floating point values in a fixed order. I also ran it twice on my laptop's CPU while building it, and those two runs matched each other exactly. Compared with the GPU, the CPU runs gave the same parameter counts and baseline outputs, but slightly different losses (r=4 1.6292 on CPU) and different wording for three of the four LoRA summaries. The main source is LoRA dropout: on the GPU its random masks come from the CUDA generator, which is a different random stream from the CPU one even with the same SEED. Small differences in the adapter weights can change a greedy decoding path after one different token, so the results are reproducible on a given device but not word for word across devices. Details are in RUN_LOG.txt.

Final findings

LoRA made fine-tuning flan-t5-small very cheap. Only 0.22% of the parameters were trained at r=4, each run took about a minute on a T4, and the saved adapter is under 700 KB, compared to roughly 300 MB for the full model. The clearest effect of fine-tuning was on output format: the base model answered with a question copied from the dialogue, and the fine-tuned model wrote a proper third-person summary in the DialogSum style. The summaries are still imperfect, with mislabeled speakers in Example 1 and a missing main decision in Example 2 for both ranks, and I have tried to say that plainly instead of overselling the results. Increasing the rank to 16 lowered the training loss a little but did not give better summaries on

these examples, so for this model size and training budget the smaller $r=4$ adapter was the more efficient choice.